INFORME PRÁCTICA 1
INTRODUCCIÓN A OPEN CV
Y ESPACIOS DE COLOR

Andrés Gil Vicente & Jorge Carnicero Príncipe

21 de septiembre de 2025

Índice

1. Preparación del Entorno de Trabajo **3**

1.1. Carga de Librerías Necesarias 3

2. Apartado A: Uso Básico de Funciones de OpenCV **3**

2.1. Tarea A.1: Carga de Imágenes 4

2.2. Tarea A.2: Visualización de Imágenes 4

2.3. Tarea A.3: Conversión a Escala de Grises 5

2.4. Tarea A.4: Redimensionamiento de Imágenes 6

2.5. Tarea A.5: Recorte del Centro de la Imagen 7

2.6. Tarea A.6: Rotación de Imágenes 9

2.7. Tarea A.7: Guardado de Imágenes 10

3. Apartado B: Segmentación por Color **10**

3.1. Tarea B.1: Carga Masiva de Imágenes 10

3.2. Tarea B.2: Funciones Auxiliares para Visualización y Guardado 11

3.3. Tarea B.3: Conversión al Espacio de Color HSV 11

3.4. Tarea B.4: Segmentación de Colores Naranjas 12

3.5. Tarea B.5: Segmentación de Colores Blancos 14

3.6. Tarea B.6: Combinación de Máscaras 15

3.7. Tarea B.7: Guardado de Resultados 16

4. Ejercicios Adicionales **17**

4.1. Pregunta B.1: Segmentación de Logotipo por Color 17

4.2. Pregunta B.2: Diferencias entre cv2.imread() e imageio.imread() 19

4.2.1. Análisis del Problema 19

4.2.2. Solución al Problema 19

1. Preparación del Entorno de Trabajo

1.1. Carga de Librerías Necesarias

Para el desarrollo de esta práctica de introducción a OpenCV y espacios de color, es fundamental cargar las librerías que proporcionan las herramientas adecuadas para el procesamiento y análisis de imágenes.

Código 1.0: Importación de librerías básicas

```
import cv2, numpy as np, matplotlib.pyplot as plt
import os
import imageio
from typing import List
from utils import get_hsv_color_ranges
import glob
```

Las librerías utilizadas cumplen funciones específicas:

- `cv2`: Librería principal de OpenCV para procesamiento de imágenes
- `numpy`: Manipulación eficiente de arrays y matrices
- `matplotlib.pyplot`: Visualización de datos e imágenes
- `os`: Operaciones del sistema operativo para manejo de archivos
- `imageio`: Lectura y escritura de imágenes en diferentes formatos
- `glob`: Búsqueda de archivos mediante patrones
- `utils`: Funciones auxiliares personalizadas para análisis de rangos de color

2. Apartado A: Uso Básico de Funciones de OpenCV

El objetivo de este apartado es aprender las funcionalidades básicas de OpenCV mediante el uso de imágenes almacenadas en la carpeta `data`. Se abordarán las siguientes tareas fundamentales:

1. Carga de imágenes
2. Visualización de imágenes
3. Conversión de espacios de color
4. Redimensionamiento de imágenes

5. Recorte de regiones de interés
6. Rotación de imágenes
7. Guardado de imágenes procesadas

2.1. Tarea A.1: Carga de Imágenes

La función `cv2.imread()` permite cargar imágenes desde el disco al espacio de memoria como matrices numéricas.

Código 2.0: Carga de una imagen desde disco

```
# Cargar una imagen usando cv2.imread()
img = cv2.imread('../data/003.jpg')
```

La función `imread()` toma como argumento la ruta de la imagen y devuelve una matriz tridimensional que representa la imagen en formato BGR (Blue-Green-Red).

2.2. Tarea A.2: Visualización de Imágenes

OpenCV proporciona la función `cv2.imshow()` para mostrar imágenes en ventanas del sistema operativo.

Código 2.1: Visualización de imagen con OpenCV

```
# Mostrar la imagen usando cv2.imshow()
cv2.imshow("First image", img)
cv2.waitKey(0)  # Esperar hasta que se presione una tecla
cv2.destroyAllWindows()  # Cerrar todas las ventanas abiertas
```

Las funciones complementarias son:

- `cv2.waitKey(0)`: Pausa la ejecución hasta recibir una entrada del teclado
- `cv2.destroyAllWindows()`: Cierra todas las ventanas de visualización abiertas

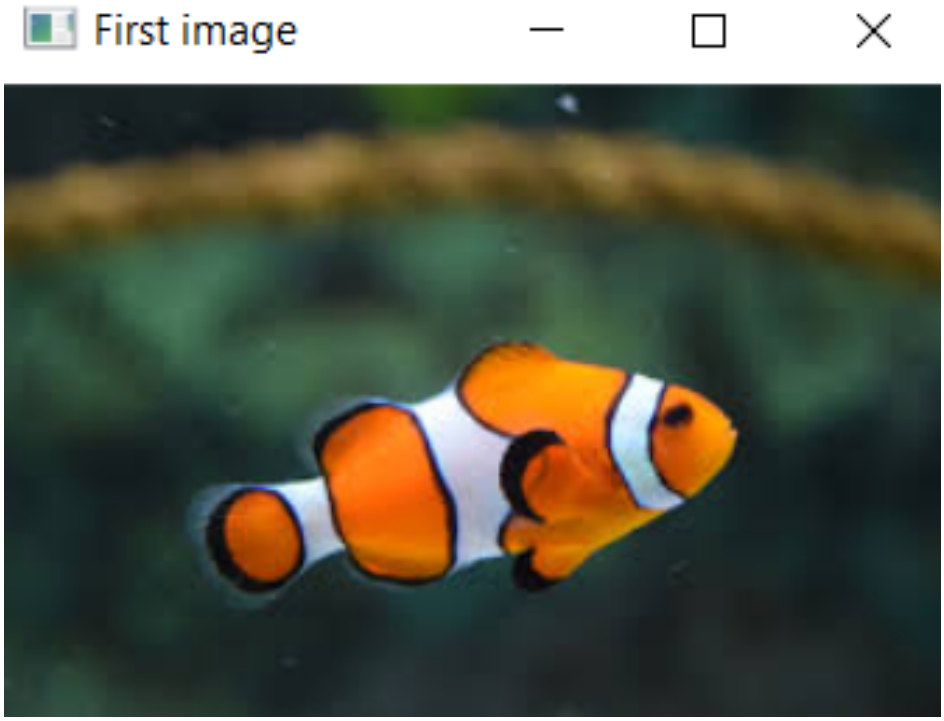


Figura 1: Ejemplo mostrar imagen 003

2.3. Tarea A.3: Conversión a Escala de Grises

La conversión de color a escala de grises es una operación fundamental en el procesamiento de imágenes.

Código 2.2: Conversión a escala de grises

```
# Convertir imagen de color a escala de grises
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Verificar las dimensiones de las imágenes
print("Color shape:", img.shape)   # (height, width, channels)
print("Gray shape:", gray.shape)   # (height, width)

# Mostrar la imagen en escala de grises
cv2.imshow("Gray image", gray)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Para pasar de RGB a escala de grises, OpenCV realiza internamente una media ponderada: $0{,}6 \cdot R + 0{,}2 \cdot G + 0{,}1 \cdot B$.

Es importante recordar que OpenCV utiliza el orden BGR en lugar del convencional RGB.

Resultado esperado:

- Imagen a color: shape (183, 275, 3)
- Imagen en grises: shape (183, 275)



Figura 2: Ejemplo mostrar figura 003 en escala de grises

2.4. Tarea A.4: Redimensionamiento de Imágenes

La función `cv2.resize()` permite cambiar las dimensiones de una imagen a un tamaño específico.

Código 2.3: Redimensionamiento de imagen

```
new_height_size, new_width_size = 200, 200 # Redimensionar a 200x200
resized = cv2.resize(src=img, dsize=(new_height_size, new_width_size))

# Verificar las nuevas dimensiones
print("Resized shape:", resized.shape)

# Mostrar la imagen redimensionada
cv2.imshow("Resized image", resized)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

La función requiere especificar el tamaño de destino como una tupla `(width, height)` en el parámetro `dsize`.



Figura 3: Ejemplo imagen 003 con tamaño redimensionado

2.5. Tarea A.5: Recorte del Centro de la Imagen

El recorte de imágenes se realiza mediante indexación de arrays, seleccionando regiones específicas de interés.

Código 2.4: Recorte de región de interés centrada

```
# Obtener dimensiones de la imagen original
h, w = img.shape[:2]
cy, cx = h//2, w//2  # Coordenadas del centro

# Definir dimensiones del recorte
new_h, new_w = 120, 160
half_of_new_h, half_of_new_w = new_h//2, new_w//2

# Calcular límites del recorte
new_top_row, new_down_row = cy-half_of_new_h, cy+half_of_new_h
```

```
new_left_col, new_right_col = cx-half_of_new_w, cx+half_of_new_w

# Recortar la imagen usando slicing
roi = img[new_top_row:new_down_row, new_left_col:new_right_col]

# Verificar las dimensiones del ROI
print("ROI shape:", roi.shape)

# Mostrar la región de interés
cv2.imshow("Cropped image", roi)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

El recorte se realiza tratando la imagen como una matriz y utilizando slicing para seleccionar las filas y columnas deseadas. ROI significa "Region of Interest" (Región de Interés).



Figura 4: Región de interés recortada del centro de la imagen (120x160)

2.6. Tarea A.6: Rotación de Imágenes

La rotación de imágenes requiere el cálculo de una matriz de rotación y su posterior aplicación a la imaegen que queremos rotar.

Código 2.5: Rotación de imagen

```
# Calcular la matriz de rotación
M = cv2.getRotationMatrix2D(center=(cy,cx), angle=90, scale=1)
print("Matriz de rotación:")
print(M)

# Aplicar la transformación afín
rotated = cv2.warpAffine(src=img, M=M, dsize=img.shape[:2])

# Mostrar la imagen rotada
cv2.imshow("Rotated image", rotated)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Los parámetros de `getRotationMatrix2D()` son:

- `center`: Punto de rotación (cy, cx)
- `angle`: Ángulo en grados (positivo significa antihorario)
- `scale`: Factor de escala (1 significa tamaño original)



Figura 5: Imagen rotada 90 grados en sentido antihorario

2.7. Tarea A.7: Guardado de Imágenes

La función `cv2.imwrite()` permite guardar imágenes procesadas en el disco.

Código 2.6: Guardado de imagen procesada

```
# Guardar la imagen en disco
save_img_path = "salida_opcionA_1.png"
cv2.imwrite(save_img_path, img) # especificamos el path de destino y la
↪  imagen a guardar
```

La función devuelve `True` si el guardado fue exitoso, `False` en caso contrario.

3. Apartado B: Segmentación por Color

El objetivo de este apartado es segmentar los colores naranja y blanco de las imágenes de los peces mediante técnicas de análisis en el espacio de color HSV. La segmentación por color es una técnica fundamental en computer vision que permite identificar y extraer objetos basándose en sus características cromáticas.

3.1. Tarea B.1: Carga Masiva de Imágenes

Para procesar múltiples imágenes de forma eficiente, se implementa un método de carga masiva.

Código 3.0: Función para cargar múltiples imágenes

```
def load_images(filenames: List) -> List:
    return [cv2.imread(filename) for filename in filenames]

# Construir lista de paths de todas las imágenes en la carpeta data
imgs_gen_path = '../data/*.jpg'
imgs_path = sorted(glob.glob(imgs_gen_path))
imgs = load_images(imgs_path)

print(f"Paths de las imágenes: {imgs_path}")
print(f"Número de imágenes cargadas: {len(imgs)}")
```

La función `glob.glob()` permite encontrar archivos que coincidan con un patrón específico, en este caso todos los archivos `.jpg` de la carpeta `data`. Resultado esperado:

```
Paths de las imágenes: ['../data/000.jpg', '../data/001.jpg', '../data/002.jpg',
                        '../data/003.jpg', '../data/004.jpg', '../data/005.jpg',
                        '../data/atleti.jpg']
Número de imágenes cargadas: 7
```

3.2. Tarea B.2: Funciones Auxiliares para Visualización y Guardado

Se implementan funciones auxiliares para estandarizar la visualización y guardado de imágenes.

Código 3.1: Funciones auxiliares

```python
def show_image(img: np.array, img_name: str = "Image"):
    cv2.imshow(img_name, img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

def write_image(output_folder: str, img_name: str, img: np.array):
    img_path = os.path.join(output_folder, img_name)
    os.makedirs(output_folder, exist_ok=True)
    cv2.imwrite(img_path, img)
```

Estas funciones encapsulan las operaciones de visualización y guardado, facilitando su reutilización y mantenimiento del código.

3.3. Tarea B.3: Conversión al Espacio de Color HSV

El espacio de color HSV (Hue, Saturation, Value) separa la información cromática de la luminancia, facilitando la segmentación por color. Por eso convertimos nuestras imágenes a este espacio de color:

Código 3.2: Conversión a espacio HSV

```python
# Convertir todas las imágenes al espacio HSV
hsv_imgs = [cv2.cvtColor(im, cv2.COLOR_BGR2HSV) for im in imgs]

# Verificar las dimensiones de las imágenes convertidas
for i, hsv in enumerate(hsv_imgs):
    print(f"La imagen {i}: {hsv.shape}")
```

En el espacio HSV:

- **H (Hue)**: Tono del color, representa el color puro
- **S (Saturation)**: Saturación, intensidad del color
- **V (Value)**: Valor o brillo, componente de luminancia

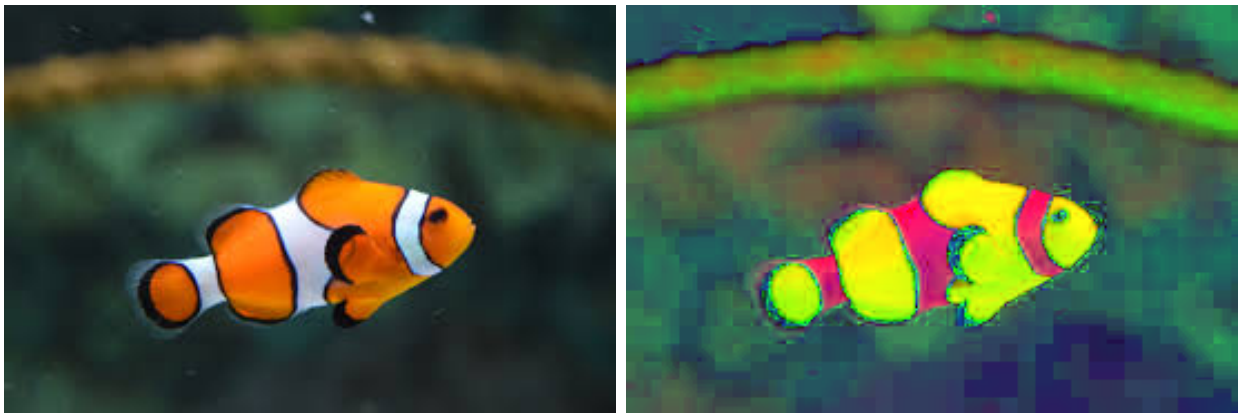


Figura 6: Comparación de imágen original y su conversión al espacio HSV

3.4. Tarea B.4: Segmentación de Colores Naranjas

La segmentación de colores naranjas se realiza definiendo rangos específicos en el espacio HSV. Esto lo hacemos tras haber estado probando valores mediante la función de `get_hsv_color_ranges()`

Código 3.3: Segmentación de colores naranjas

```
# Definir el rango de colores naranjas en HSV
light_orange = (0, 200, 120)   # Límite inferior
dark_orange = (255, 255, 255)  # Límite superior

# Crear máscaras para colores naranjas
orange_masks = [cv2.inRange(hsv_im, light_orange, dark_orange)
                for hsv_im in hsv_imgs]

# Aplicar las máscaras a las imágenes originales
orange_segmented = [cv2.bitwise_and(img, img, mask=mask)
                    for img, mask in zip(imgs, orange_masks)]

# Visualizar resultados para una imagen específica
n = 3
show_image(imgs[n], "Original image")
show_image(orange_masks[n], "Orange Masks")
```

```
show_image(orange_segmented[n], "Orange segmented image")
```

La función `cv2.inRange()` verifica si cada píxel de la imagen HSV está dentro del rango de colores especificado, devolviendo una máscara binaria o booleana donde:

- **255 (True)**: El píxel está dentro del rango
- **0 (False)**: El píxel está fuera del rango

La función `cv2.bitwise_and()` aplica la máscara a la imagen original, conservando solo los píxeles donde la máscara es verdadera.



Figura 7: Proceso de segmentación de colores naranjas: (a) Imagen original, (b) Máscara naranja, (c) Imagen segmentada

3.5. Tarea B.5: Segmentación de Colores Blancos

Siguiendo el mismo procedimiento que en el proceso anterior, se segmentan los colores blancos definiendo rangos apropiados en HSV.

Código 3.4: Segmentación de colores blancos

```
# Definir el rango de colores blancos en HSV
light_white = (0, 0, 180)
dark_white = (170, 175, 255)

# Crear máscaras para colores blancos
white_masks = [cv2.inRange(hsv_im, light_white, dark_white)
               for hsv_im in hsv_imgs]

# Aplicar las máscaras a las imágenes originales
white_segmented = [cv2.bitwise_and(img, img, mask=mask)
                   for img, mask in zip(imgs, white_masks)]

# Visualizar resultados para una imagen específica
n = 3
show_image(imgs[n], "Original image")
show_image(white_masks[n], "White Mask")
show_image(white_segmented[n], "White segmented image")
```

La segmentación de blancos requiere considerar que el color blanco en HSV tiene:

- Baja saturación (cercana a 0)
- Alto valor de luminancia (cercano a 255)
- El tono puede variar ampliamente



Figura 8: Proceso de segmentación de colores blancos: (a) Imagen original, (b) Máscara blanca, (c) Imagen segmentada

3.6. Tarea B.6: Combinación de Máscaras

Para segmentar objetos que contengan tanto colores naranjas como blancos, se combinan ambas máscaras utilizando operaciones lógicas de `OR` y de `AND`.

Código 3.5: Combinación de máscaras naranja y blanca

```
# Combinar máscaras usando OR lógico
fish_masks = [cv2.bitwise_or(o,w) for o, w in zip(orange_masks,
↪  white_masks)]

# Aplicar la máscara combinada a las imágenes originales
fish = [cv2.bitwise_and(img, img, mask=mask)
        for img, mask in zip(imgs, fish_masks)]

# Visualizar resultados
n = 3
show_image(imgs[n], "Original image")
show_image(fish_masks[n], "Fish Mask")
```

```
show_image(fish[n], "White & Orange segmented image")
```

La operación `cv2.bitwise_or()` implementa la puerta lógica OR, donde:

- Si un píxel es naranja **O** blanco, se incluye en la máscara final
- Permite segmentar objetos que contengan cualquiera de los dos colores



Figura 9: Combinación de máscaras: (a) Imagen original, (b) Máscara combinada, (c) Segmentación final

3.7. Tarea B.7: Guardado de Resultados

Los resultados de la segmentación se guardan sistemáticamente para su posterior análisis.

Código 3.6: Guardado masivo de imágenes segmentadas

```
# Definir carpeta de salida y guardar todas las imágenes de peces
output_folder = "output_folder"
for i in range(len(fish)):
    img_name = f"fish_{i}.png"
```

```
write_image(output_folder=output_folder, img_name=img_name, img=fish[i])
```

Se genera una colección de imágenes segmentadas numeradas secuencialmente para facilitar su organización.

4. Ejercicios Adicionales

4.1. Pregunta B.1: Segmentación de Logotipo por Color

Se aplican las técnicas aprendidas para segmentar el escudo del Atlético de Madrid, descomponiéndolo en sus tres colores principales: rojo, blanco y azul.

Código 4.0: Segmentación del escudo del Atlético de Madrid

```
# Cargar imagen del escudo del Atlético
img = cv2.imread("../data/atleti.jpg")

# Convertir a espacio HSV
hsv_img = cv2.cvtColor(src=img, code=cv2.COLOR_BGR2HSV)

# Verificar carga correcta
cv2.imshow("escudo", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Código 4.1: Definición de rangos de color para el escudo

```
# Rangos para detectar el color rojo del escudo
red_hsv_lower = (0, 228, 182)
red_hsv_upper = (255, 255, 255)
red_mask = cv2.inRange(hsv_img, red_hsv_lower, red_hsv_upper)

# Rangos para detectar el color blanco del escudo
white_hsv_lower = (5, 0, 215)
white_hsv_upper = (212, 147, 255)
white_mask = cv2.inRange(hsv_img, white_hsv_lower, white_hsv_upper)

# Rangos para detectar el color azul del escudo
blue_hsv_lower = (110, 232, 56)
blue_hsv_upper = (130, 255, 255)
blue_mask = cv2.inRange(hsv_img, blue_hsv_lower, blue_hsv_upper)
```

Código 4.2: Combinación final de máscaras del escudo

```
# Combinar máscaras azul y roja
blue_and_red_mask = cv2.bitwise_or(red_mask, blue_mask)

# Crear máscara final combinando todos los colores
final_mask = cv2.bitwise_or(blue_and_red_mask, white_mask)

# Aplicar máscara final al escudo
escudo = cv2.bitwise_and(img, img, mask=final_mask)

# Visualizar y guardar resultados
show_image(img, "Escudo Original")
show_image(final_mask, "Máscara Final")
write_image(output_folder="output_folder",
            img_name="escudo_final_mask.png", img=final_mask)
show_image(escudo, "Escudo Segmentado")
write_image(output_folder="output_folder",
            img_name="escudo_maskeado.png", img=escudo)
```



Figura 10: Segmentación del escudo del Atlético de Madrid: (a) Escudo original, (b) Máscara combinada, (c) Escudo segmentado

4.2. Pregunta B.2: Diferencias entre cv2.imread() e imageio.imread()

Se investiga el comportamiento diferencial entre las funciones de carga de imágenes de OpenCV e imageio.

Código 4.3: Carga con imageio y visualización con OpenCV

```
def load_images(filenames: List) -> List:
    return [imageio.imread(filename) for filename in filenames]

imgs_path = ["../data/*.jpg"]
imgs_path = glob.glob(imgs_path[0])

# Cargar imágenes con imageio
question_imgs = load_images(imgs_path)

# Mostrar imágenes cargadas con imageio
for i, im in enumerate(question_imgs):
    show_image(im, f"Question image {i}")
```

4.2.1. Análisis del Problema

Al cargar una imagen con `imageio.imread()`, los valores de los píxeles se almacenan en formato **RGB**. Sin embargo, cuando la mostramos con `show_image()` (que internamente utiliza `cv2.imshow()`), OpenCV interpreta las imágenes en formato **BGR**.

Esta diferencia en el orden de los canales provoca que las imágenes aparezcan con colores distintos a los originales:

- Los canales rojo y azul se intercambian
- Los peces que antes eran naranjas ahora aparecen azulados
- Se mantienen los colores blancos ya que no dependen del orden de canales

4.2.2. Solución al Problema

Para corregir este comportamiento, se debe convertir explícitamente el espacio de color antes de la visualización:

Código 4.4: Corrección del problema RGB/BGR

```
# Convertir de RGB a BGR para compatibilidad con OpenCV
img_bgr = cv2.cvtColor(question_imgs[0], cv2.COLOR_RGB2BGR)
show_image(img_bgr, "Example image corrected")
```



Figura 11: Comparación entre carga con imageio (RGB) y visualización con OpenCV (BGR): (a) Colores incorrectos, (b) Colores corregidos